Smart HVAC System Lab - Final Project

Course: CPRE 2870X Fundamentals of Cyber-physical Systems
Instructors: Dr. Cheng Wang & Iris Top
Date: May 15, 2025
Authors: Blane Fullmer, Micah Scheaffer, and Jacob Sterenberg

Table of Contents

Project Viewable on Notion web version here

Project Summary

Back to Main Page	Smart HVAC System Lab - Final Project
Next Page	🛠️ Problem and Design

Purpose & Methods

The goal of this project is to implement software to automate a model smart-home HVAC system. Throughout this course, we have learned about the Cyber-physical Systems, which enable IoT (Internet of Things) systems to function, including:

- Foundations of Electrical Engineering
  - Low-level circuit analysis
  - Analogue to digital conversion
  - Basic signal processing
- Integrated computing
  - Digital logic
  - Microcontroller Units
  - Memory allocation
- Software Development
  - Basic python programming
  - Circuit python
  - Software environment setup
  - Cloud environment setup
  - Network communication

Throughout this course in labs we have been tinkering with breadboard circuits, various sensors, the RaspberryPi3, the Adafruit FeatherS2 and the Adafruit FunHouse to gain a deeper understanding of the systems used in the project.

Our final project represents the sum of all of these principles that we have learned throughout the course, which is documented here.

Problem and Design

Back to Main Page	Smart HVAC System Lab - Final Project
Next Page	📋 Case Study
Previous Page	⚡ Project Summary

System Purpose

We are developing a cyber-physical system to regulate the heating and cooling of a home. This fulfils an educational purpose, but emulates a real-world challenge an engineer in the HVAC Industry may have. In a typical HVAC system, there is one thermostat that is centralized in a house with one temperature measurement node. This standard design only allows for one temperature to be set and doesn't provide accurate readings for the rooms of the house. In the system that was created, there is an individual sensor in each room of the house and a UI that allows each room to be set at different temperatures, or the same if preferred.

System Components

Testbed

The HVAC System operates on the testbed, which functions as model smart-home. The testbed has three rooms, one large and two small, a mini HVAC unit, and piping from the HVAC to each room.

Each enclosed room has a temperature measurement node, which is an Adafruit Funhouse board connected to an LM35 temperature measurement sensor. The HVAC unit is connected to the primary control node, which can communicate with our server. There are dampers in the pipes leading to each room, which can allow or restrict airflow to each room from the HVAC system using a servo controlled by the secondary node.

Each node can communicate with each other by connecting to our server through wifi.

This allows the system to:

- Receive user input from a NODE-RED dashboard
- Adjust the temperature change induced by the HVAC unit based on user input and the temperatures measured in each room
- Adjust the airflow to each room based on the target temperatures of the rooms and the thermal output of the HVAC system.

For the physical system to correctly functionally heat and cool the model home, multiple layers of software needed to be implemented. This includes network communication, temperature measurement, control logic, and actuation.

Network Communication & Dashboard

The system's nodes communicate through an MQTT broker, which acts as a central server for publishing and retrieving data. Node-RED has a flow-based program that provides visual feedback through a dashboard interface. Users can interact with specific topics through this dashboard, such as adjusting thermostat settings or enabling manual mode. On startup, the primary node subscribes to feeds for temperatures, dampers, and HVAC control. When new topics are published to the broker, a message-receiving function parses the information and updates the relevant lists or variables. This enables the primary node to make real-time decisions about heating and cooling needs for each zone. There is also a secondary node that communicates to the primary via a socket connection. This entire communication system relies on a proper configuration of IP addresses and broker-client connections in the networking setup.

Temperature Measurement

Each room contains a Funhouse device connected to an LM35 temperature sensor, which provides voltage readings proportional to temperature. The temperature measurement code uses the LM35's technical specifications to convert voltage readings to both Celsius and Fahrenheit. The Funhouse stores this value until a new measurement differs by more than 0.5 percent from the previous reading, at which point it publishes the zone's temperature to the broker.

Control Algorithm

With multiple thermostats and temperature sensors, different rooms could simultaneously request heating and cooling. The system uses an automatic control algorithm that prioritizes rooms based on their temperature difference from setpoint. When multiple rooms need conditioning, the system ensures efficient operation by coordinating the HVAC unit and dampers to serve rooms with similar needs while preventing simultaneous heating and cooling.

Actuation

The primary node receives temperature setpoints from the user, actual temperatures from the measurement nodes, and then calculates appropriate actions for the HVAC and the corresponding damper positions. This information is sent to the actuation script, which is relayed to the secondary node to initiate the HVAC and Dampers.

Automatic Control logic

The model HVAC system that we implement takes into consideration three “zones”, or rooms. The user is allowed to set the thermostat of the individual zones to different values, and our control algorithm must drive the temperatures of the rooms to those desired “set points”. The major constraint is that the heater and cooler cannot both be active at the same time. This leads to an obvious problem that our controller needs to automate: What happens if one room needs to be cooled, and another needs to be heated?

Zone Priority and The Decision Flag

To overcome this ‘one or the other’ dilemma, our system calculates a **priority** for each room. This is done by taking the absolute value of the current temperature minus the set temperature. Once this has been done for each zone, a **Decision Flag**, named HVAC, is set to either Heat, Cool, or Off. The decision flag remains set to a non-zero value until the setpoint and current temperature of the highest priority zone are within ±.5°F.

The following code snippet from the control loop encapsulates the basic **Decision Process**

```
if HVAC == 0:
        print(current_temp)
     for zone in range(num_zones):
            ##### PRIORITY LIST #####
              priority_list[zone] = abs(current_temp[zone] - thermostat_List[zone])
            actuation.set_damper(zone,0)
            networking.mqtt_publish_message(networking.IN_DAMPER_FEEDS[zone
    #print(f"THE PRIORITY LIST: {priority_list}")
    max_zone = priority_list.index(max(priority_list))
```

The variable **max_zone** is the index of the highest priority. This is later used to decide whether to set the HVAC flag to 2 for heat, 1 for cool, or 0 to remain off. This control logic proved very effective for ensuring that the system chose to heat or cool in the most efficient way possible.

Damper control

Consider a scenario where the decision block has decided that the system needs to heat in order to satisfy the highest priority zone. It would be inefficient to pump hot air into a room that needs to be cooled, so naturally the damper to that room will be shut, preventing unnecessary direct heating for that zone. However, what if another zone of a lower priority also needs to be heating? In the event that more than one zone wants to be brought in the same direction, our dampers are automated to optimize this airflow to each room.

In our code, damper logic is encapsulated by the function **damp_control()**. It takes two arrays as inputs, containing all zones that want to be open to the heat or cool, and another array of zones that should remain closed. For all zones in the open list, the function calculates a gradient in steps of 20, with a minimum of 50, based on the same priority calculation based on the Decision Process.

The damper setting is calculated using a constrained proportional control loop (P-loop), which sets the damper position based on the temperature difference between the current and desired values. The loop is constrained in two ways: it sets a minimum damper opening of 50%, and it rounds the output to the nearest 20% increment.

This ensures that the vents are opened to all zones that want the conditioned air, but keeps it proportional to their need. The highest priority will always have it's damper opened the most.

The code snippet is provided here:

```
def damp_control(open_list, closed_list):
    global current_temp
    global thermostat_List

        #Opens dampers to all zones that want conditioned air, proportional to their
    for number in open_list:
        actuator = (abs(current_temp[number] - thermostat_List[number]) - 1)/5*10
        actuator = round(min(actuator, 100), -1)
        actuator = round(max(actuator, 50), -1)
        actuation.set_damper(number, actuator)
        print(f"Priority for open zone {number}: {priority_list[number]}")

        #Keeps all opposing zones closed.
    for number in closed_list:
        actuation.set_damper(number, 0)
        #print(f"Priority for closed zone {number}: {priority_list[number]}")
```

Manual Mode

Purpose

This system needs a manual mode in order to test and verify the operation of systems at a lower level than the logic programmed to automate the system.

Methods

We added a UI switch node to the Node-RED dashboard. This is connected to switch nodes, which can show and hide groups when connected to a UI control node.

We set up groups for manual just mode, just automatics, and left the majority visible at all times. The groups affected by the switch are:

- System state menu
- Damper set points
- Temperature set points

Manual mode disables the temperature setpoints and enables the system state menu and the damper set points. Using the System state menu, the user can control whether the system is heating, cooling, just blowing, or off. Using the Damper set points, the user can open, close, or set somewhere in between the servos, which control airflow to each room for heating and cooling.

On the other hand, when manual mode is off, the user can set the temperature for each room, and the automatic control logic will heat or cool the rooms to the set temperatures in the process it is programmed to do.

Results

The manual mode setup allows us to control elements of the systems that the user does not normally directly control. When manual mode is engaged, it prioritizes controlling the system as opposed to accurately or efficiently achieving a desired temperature for each room.

Case Study

Back to Main Page	Smart HVAC System Lab - Final Project
Next Page	Cybersecurity Analysis
Previous Page	Problem and Design

Case Study Procedure:

In order to ensure that our control system's logic was functioning properly, we connected the primary and secondary nodes to a **Simulation**, where they would read and manipulate fake temperature values. The following is a write-up explaining moments.

The implementation of the primary control logic is visible on the two attached graphs below. We hope that this visualization can help to better explain the logic behind our control system.

First Decision: Heat zones 1 and 2, with priority to 2

When the simulation begins, the zone temperatures and their corresponding set points are as written:

Zone:	Temperature (F)	Set Temperature (F)
1	67	70
2	62	70
3	75	70

Once these values are set, the system makes a decision based on the furthest distance between current and desired temperature. In this case, Zone 2 has the highest priority, with a difference of 8. Zone 2 needs to be heated, and so the heat is set to on.

Due to the logic of the automatic dampers, vents are open to both zones that need heat, and so on the temp chart, you see Temp 1 and Temp 2 both trend upwards. On the damper chart, you can see the moment that the call was made to the dampers to open 1 and 2. Zone 2 has the higher priority, so it's damper was opened wider than Zone 1, and proportionaly lowered over time.

The moments that the set temperatures were reached by zone 1 and 2 are marked on the temperature graph by the blue and red stars respectively.

Second Decision: Cool zone 3

Almost immediately after the red star, The next priority check is made. The state of the zone temps and set points is now as follows:

Zone:	Temperature (F)	Set Temperature (F)
1	70	70
2	70	70
3	75	70

From this information, the second decision is made. Zone three is the only zone with a non zero priority, meaning it is the highest. The system is set to cool, and the yellow line can be seen visibly trending downwards. The moment this happens, the damper to zone 3 is set to open. As it cools closer to its set point, the damper slowly closes as to prevent overshooting the zone. This is visible as the cluster of yellow bars on the damper graph.

The moment that zone 3 reached the set point is marked with the yellow star.

Decision 3 and 4: New Set Points

Shortly after the last zone reached its set point, all set points were changed, as shown in the table:

Zone:	Temperature (F)	Set Temperature (F)
1	70	71
2	70	72
3	70	67

The set points were not changed all at once. The first set point was first, followed by the second, etc. This second round of desired temperatures actually reveals a feature of our logic, which we will now discuss.

Once the HVAC flag is set to a nonzero value, it remains set until the zone that set it has reached its desired outcome. In this case, for a split second, the setpoints were technically 71, 70, and 70, meaning that zone 1 was given the flag, even though its set point was only a degree away. **However**, once zone 2's setpoint was changed, its dampers immediately opened to receive some of the conditioned air that it needed.

The moment that Zones 1 and 2 reached their setpoints is marked by the blue and red diamonds, respectively.

Lastly, zone 3 was cooled again down to 67. The moment it reached it's set point is marked with the Yellow Diamond.

Temperature Graph

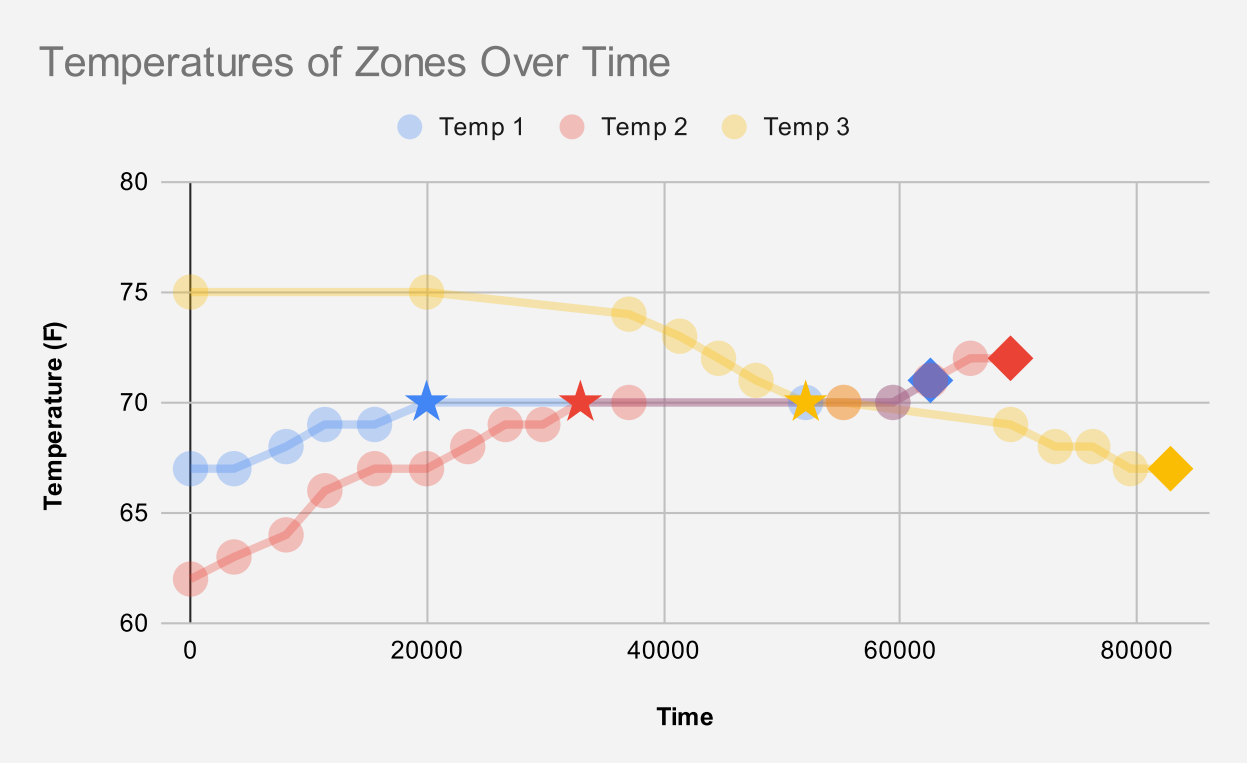

Figure 1: Reported Temperatures over Time

Please note: Temperature Values are only published when they change significantly. The trend line between points is not necessarily representative of the true values. Reported values are marked by the transparent circles.

Damper Graph

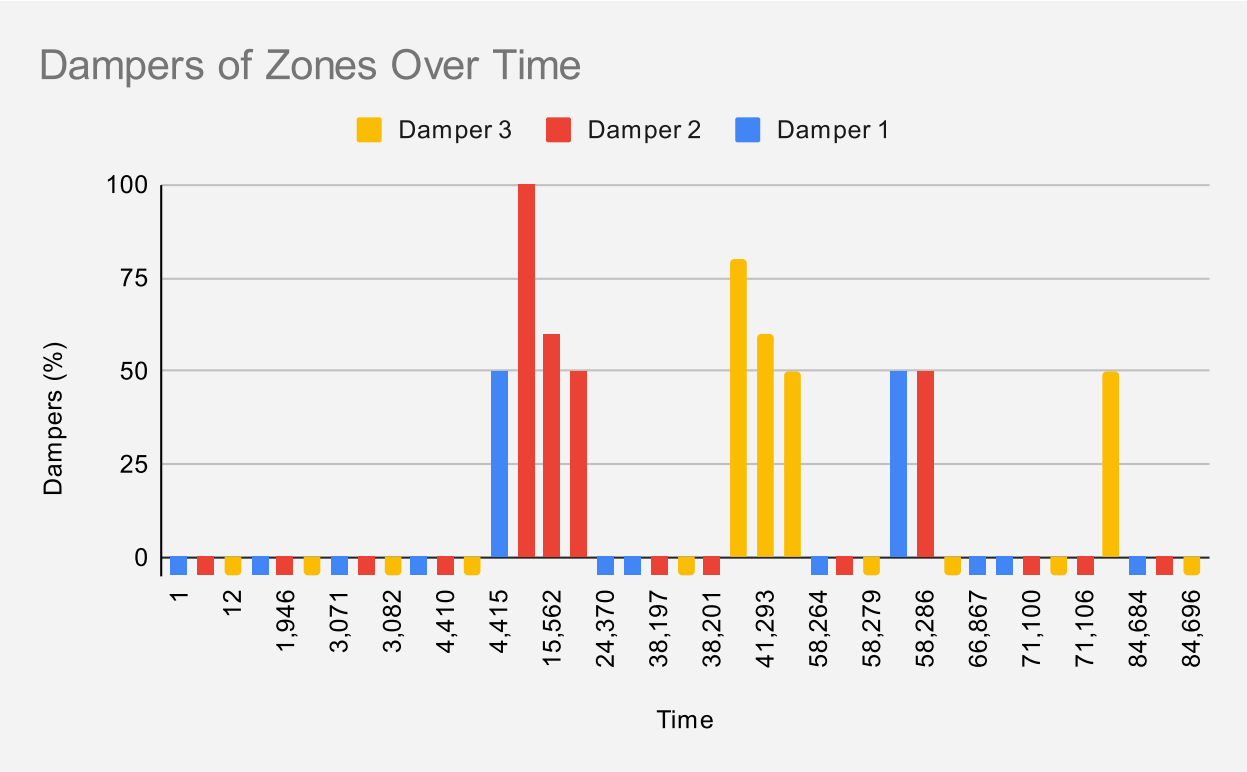

Figure 2: Damper Set Commands Sent over time

Please note: Damper values are only published when they change significantly. This graph represents the time the damper was set, and what value it was set to. Also, values of zero are graphed here as -5 to improve legibility.

Cybersecurity Analysis

Back to Main Page	Smart HVAC System Lab - Final Project
Next Page	🔎 Conclusion
Previous Page	📋 Case Study

Cybersecurity Analysis

Cybersecurity is important for any cyber-physical systems that with network communication. A bad actor with the ability to tap into a system like this could bug, disrupt, or even control the system without proper measures in place. For a system like this, the most important thing is limiting the ability to interact with any part of the system to just the user and other necessary components in the system. Many of the cybersecurity features are not implemented for the scope of this project, but it is important to be aware of the factors that could affect our system.

Jammers

If a jammer was used on the HVAC system and there were multiple wireless connections, it could be possible for the system to continue the previous task it was doing without ever turning off. To prevent wireless networking jamming, we could implement a wired connection when possible. When this isn't possible, multiple access points could be used to provide redundancy to the system. Using a heartbeat between the wireless systems provides confirmation that the systems are still connected. If this heartbeat was interrupted, the system could default to off until connection is re-established. Other measures could also be taken such as automatically switching frequencies if the system has been disconnected for too long

Eavesdropping

To prevent eavesdropping, we could implement two-way data encryption, where each party involved has their own encryption/decryption key. This makes the data being listened to useless unless the listener can decode the data.

Impersonation

Impersonation of the MQTT could allow potential attackers to gain access to topics and information being published. This could let attackers manipulate data that is necessary for the system to run. To prevent this, clients should have their own passwords and only be allowed access to the feeds/topics necessary for operation.

Impersonation of the primary control node would allow for false system commands to be published and implemented by the system. To prevent this, the primary commands could be encrypted so that only the commands that could be properly decrypted by the other nodes would be implemented. Wired connections from the primary could also help to verify the commands by posting back to the broker for confirmation.

Impersonation of the secondary control node would allow for HVAC systems to be turned on or off without the command of the primary node. This could be detrimental to inhabitants depending on the time of year. Limiting access of the secondary node to the primary only and establishing wired connections could help limit impersonation. Command confirmations could also help to ensure that the system is properly receiving the command. The heartbeat established could also determine which secondary control node IP is the correct one.

Impersonation of the temperature measurement node would allow for another system to post incorrect temperatures, confusing the primary control logic. Encrypting the data so that only the primary could decrypt it could allow the primary to verify that the data received was from the correct source.

Conclusion

Back to Main Page	Smart HVAC System Lab - Final Project
Previous Page	Cybersecurity Analysis

Key Concepts

The hardware and communication uses we learned about through this project, and in the class overall, taught us the viability and practicality of cyber-physical systems.

Hardware

Our system brought together many physical pieces: temperature sensors, servos, multiple types of microcontrollers, and single-board computers. To get all of these parts to function together, we used network communication and digital-to-analog conversion. The diversity of electronic devices used are fundamental to both the design and success of our HVAC system. Ultimately, leveraging different types of hardware expands the possibilities of what such systems can achieve.

Communication

We built our system's communication architecture around MQTT, a messaging protocol made for real-time communication between embedded devices. MQTT enabled our components to exchange data quickly and reliably over the network, bridging the gap between sensing, computation, and physical actuation. In modern electronics, network communication is essential for coordinating distributed systems, especially in applications like home automation seen here, as well as automated manufacturing, smart agriculture, and energy management.

Lessons

Through this project, we gained valuable hands-on experience in system troubleshooting. One of the most important lessons was learning to effectively debug both hardware and multiple layers of software components for a complex system. We also learned the importance of thorough testing, especially when working with distributed systems, because issues can be difficult to isolate.

Impact

For the three of us students of Engineering, this class will have an invaluable impact by building our knowledge base in such a niche area of engineering. It will also change how we perceive the electronics we use daily and take for granted, because many are not just simple devices, but complex systems that have require careful design and integration similar to this project. Such knowledge and appreciation will influence how we approach future engineering problems.

Improvements

System Design

The Smart home testbed is a good model for education, but it is not without its flaws. One thing our group noticed when using the test bed was leakage. When we were heating one room others would be impacted to a small degree. In addition, given enough time, with the HVAC off, the temperatures of the individual rooms would equalize with the surroundings. We saw the effects of this (a lot more quickly than you would in a real house) after all the rooms came to equilibrium, and the HVAC system continued to cycle on and off in short bursts to maintain the temperature.

To improve the efficiency of the system, better dampers and insulation in the rooms could be used.

Software Implementation

Our Automatic control logic uses a constrained P-loop to control the damper settings based on the heating/cooling needed. To improve the efficiency of our system, we could use the full range of the damper settings and implement a PID-loop. The lower damper settings were dismissed as rather ineffective, but fine-

tuning the system to use all the possible damper settings would allow for better temperature regulation, especially when the temperature is near the target. A correctly tuned (critically damped) PID-loop for the dampers would be able to minimize inefficiencies in the system. A PID treats the damper setting in relation to the temperature as a second-order differential equation. With known testing data on how the system reacts to heating and cooling, the PID can be tuned to reach a steady state equilibrium at the desired temperature faster. Although for our applications, a PID loop may be overkill because of the relatively broad tolerance for any given temperature set point.